

上海科技大学
ShanghaiTech University

# Filtering

CS172 Computer Vision I

Instructor: Jiayuan Gu

# Denoising is essential in image processing




Figure from http://www.gazingcat.com/tag/noise/

# Smoothing is widely used in your camera app



# Outline

- Roadmap
  - Convolution and correlation
  - Smoothing and sharpening
  - Denoising

- Learning Objectives
  - Compute a local filter response
  - Distinguish convolution and correlation
  - Predict filter behavior from a kernel
  - Explain Gaussian and median filtering
  - Choose a filter based on noise type

# Types of Images

## Binary



$I \in \{0,1\}^{H \times W}$

## Gray Scale



$I \in \{0,1,2,\ldots,255\}^{H \times W}$

$I \in \mathbb{R}^{H \times W}$

## Color



$I \in \{0,1,2,\ldots,255\}^{H \times W \times 3}$

$I \in \mathbb{R}^{H \times W \times 3}$

For simplicity, we focus on single-channel gray-scale images in this lecture

Figure from Ulas Bagci @ UCF

# Convolution and Correlation

Local filtering: how can one output pixel depend on its neighbors?

# Motivation: Image Denoising



How can we reduce noise in a photograph?

Credit to Lana Lazebnik

# Computing Average

- Let's replace each pixel with a **weighted average** of its neighbors
- The weights are called the **filter kernel**
- What are the weights for the average of a 3x3 neighborhood?

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

"box filter"

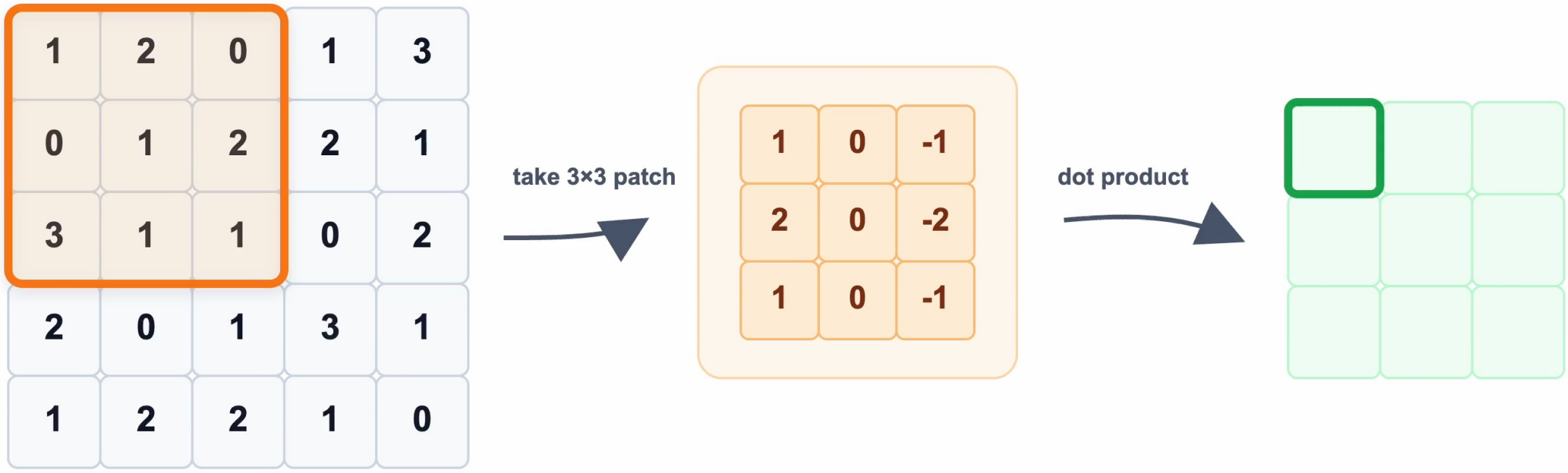
For an averaging filter, weights sum to 1, so constant regions remain unchanged

Source: D. Lowe

# Convolution

- Let $f$ be the image and $g$ be the kernel
- The convolution between $f$ and $g$ is denoted $f * g$

$$(f * g)[x, y] = \sum_{i,j} f[x - i, y - j]g[i, j]$$

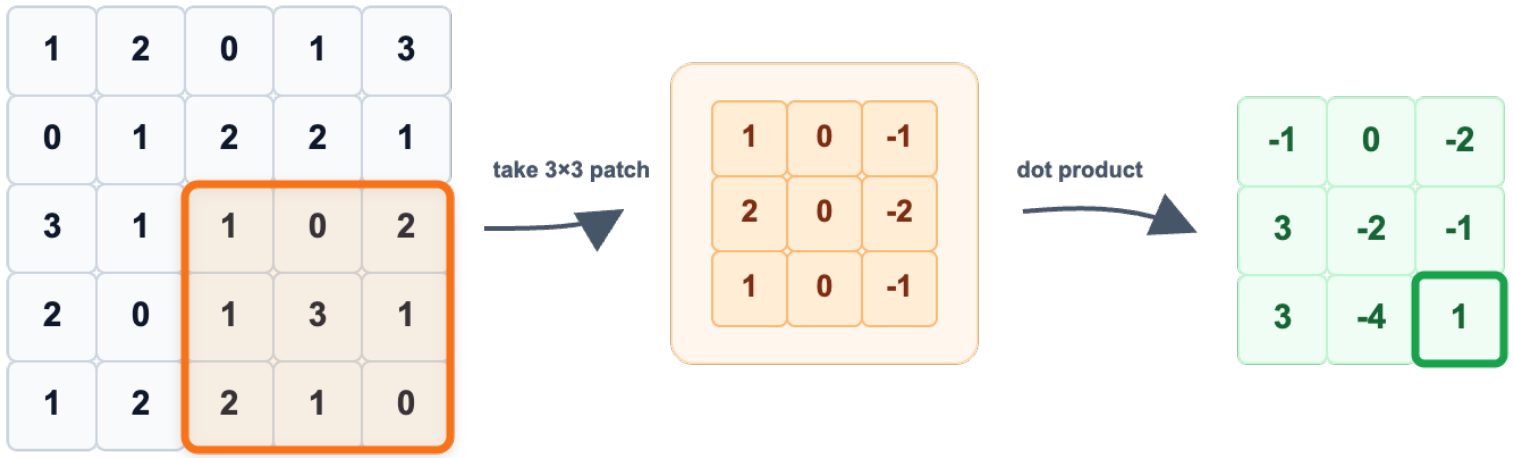


# Cross-Correlation

- The cross-correlation between $f$ and $g$ is denoted $f \otimes g$

$$(f \otimes g)[x, y] = \sum_{i,j} f[x + i, y + j] g[i, j]$$

- Note that the kernel is flipped in convolution
- Many deep-learning libraries may call cross-correlation “convolution”

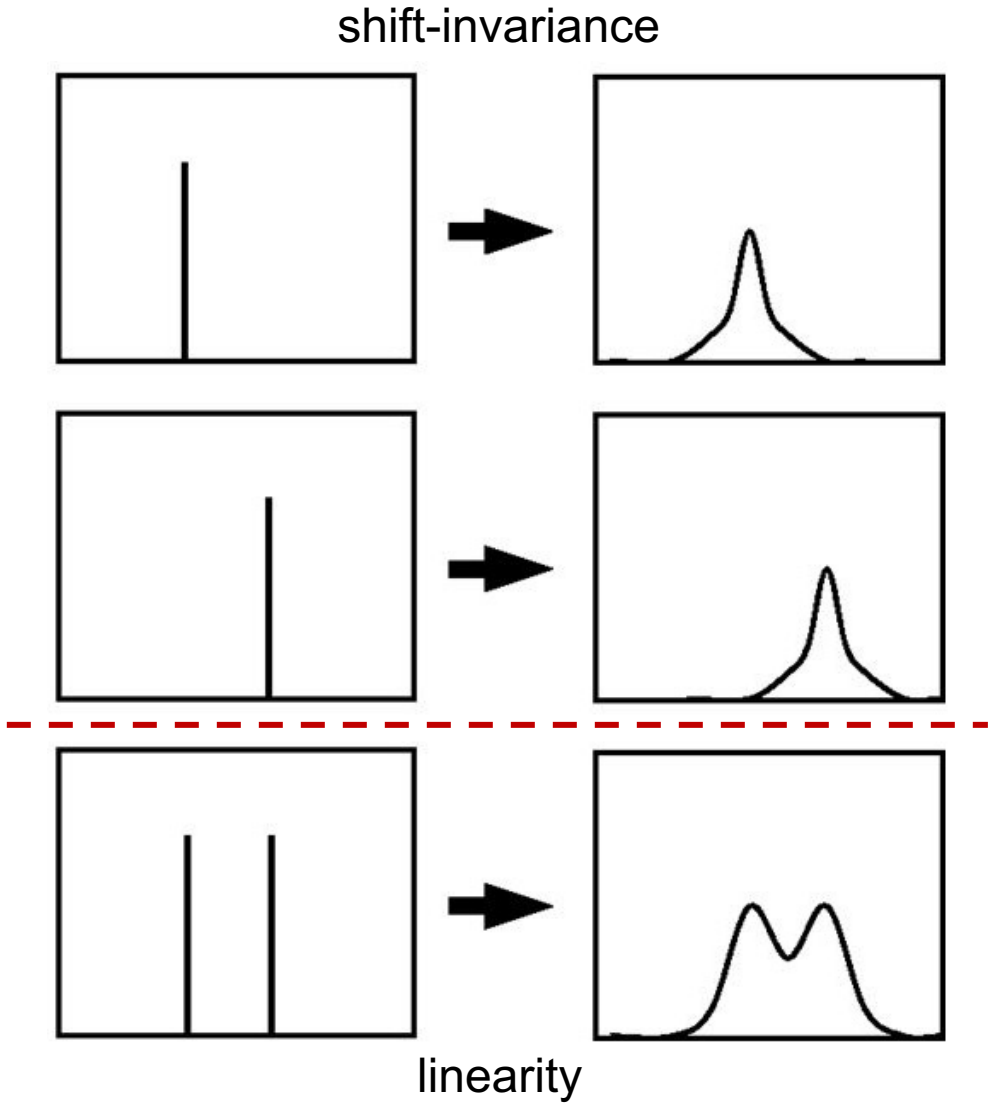


# Key Properties of Linear Filtering

- **Shift invariance:** same behavior regardless of pixel location:
  filter(shift(*f*)) = shift(filter(*f*))

- **Linearity:**
  filter($f_1 + f_2$) = filter($f_1$) + filter($f_2$)

- Theoretical result: any linear shift-invariant operator can be represented as a convolution




Credit to Lana Lazebnik

# More Properties

- Commutative: $a * b = b * a$
  - Conceptually no difference between filter and signal

- Associative: $a * (b * c) = (a * b) * c$
  - Often apply several filters one after another: $(((a * b_1) * b_2) * b_3)$
  - This is equivalent to applying one filter: $a * (b_1 * b_2 * b_3)$

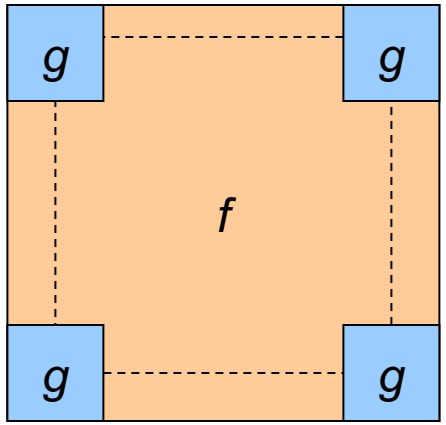
- Distributes over addition: $a * (b + c) = (a * b) + (a * c)$
- Scalars factor out: $ka * b = a * kb = k (a * b)$
- Identity: unit impulse $e = [\ldots, 0, 0, 1, 0, 0, \ldots]$, $a * e = a$

Credit to Lana Lazebnik

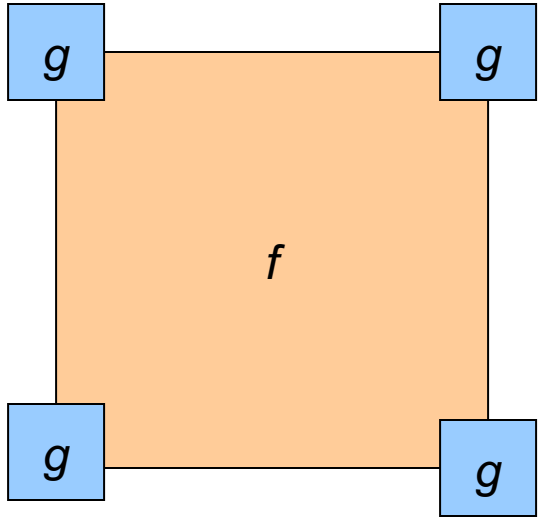
# Dealing with Edges

If we convolve image *f* with filter *g*, what is the size of the output?

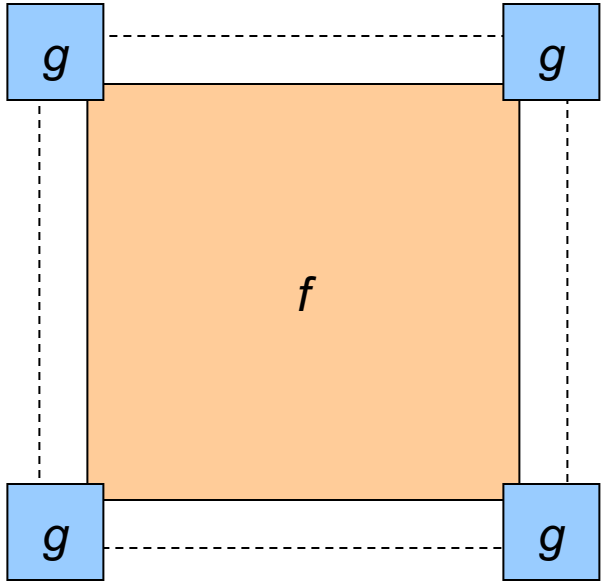
Output is smaller than input




Output is same size as input



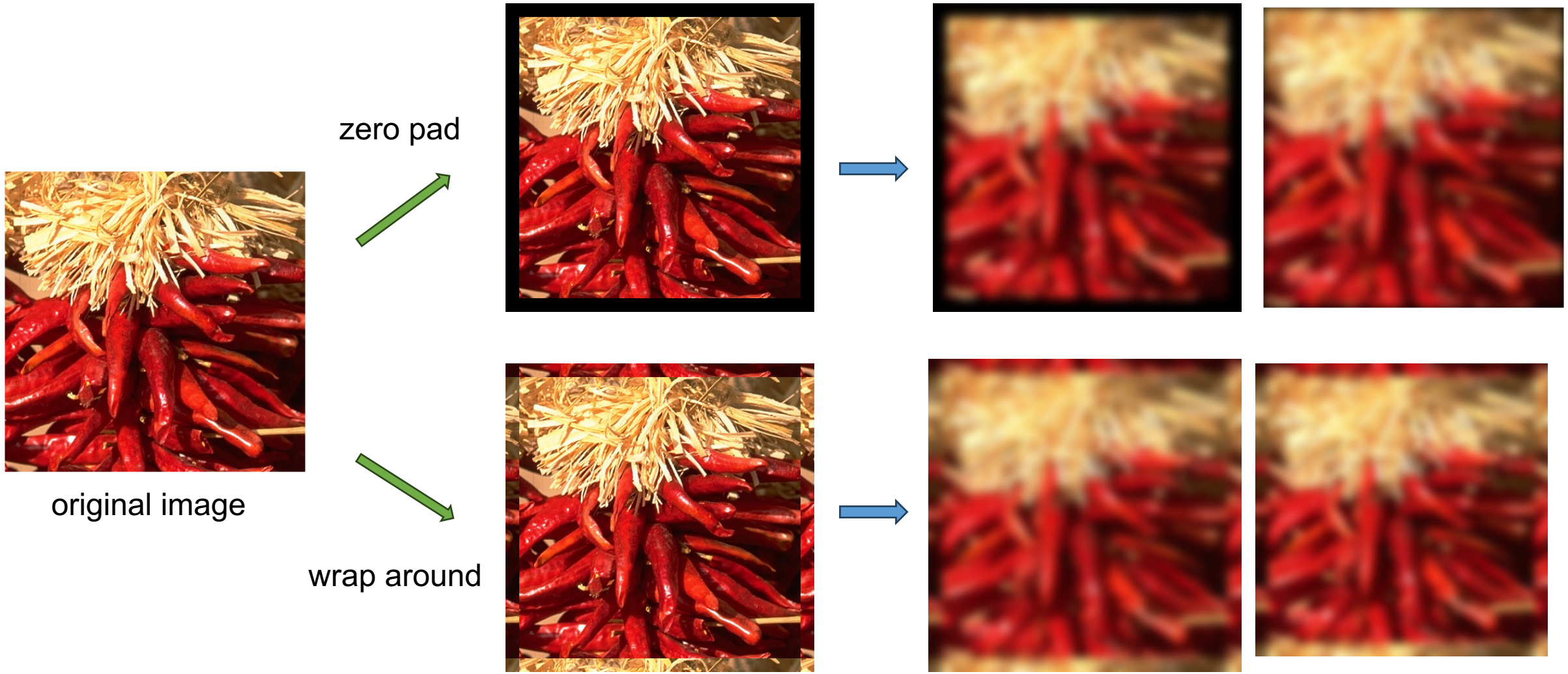

Output is larger than input




Credit to Lana Lazebnik

# Dealing with Edges

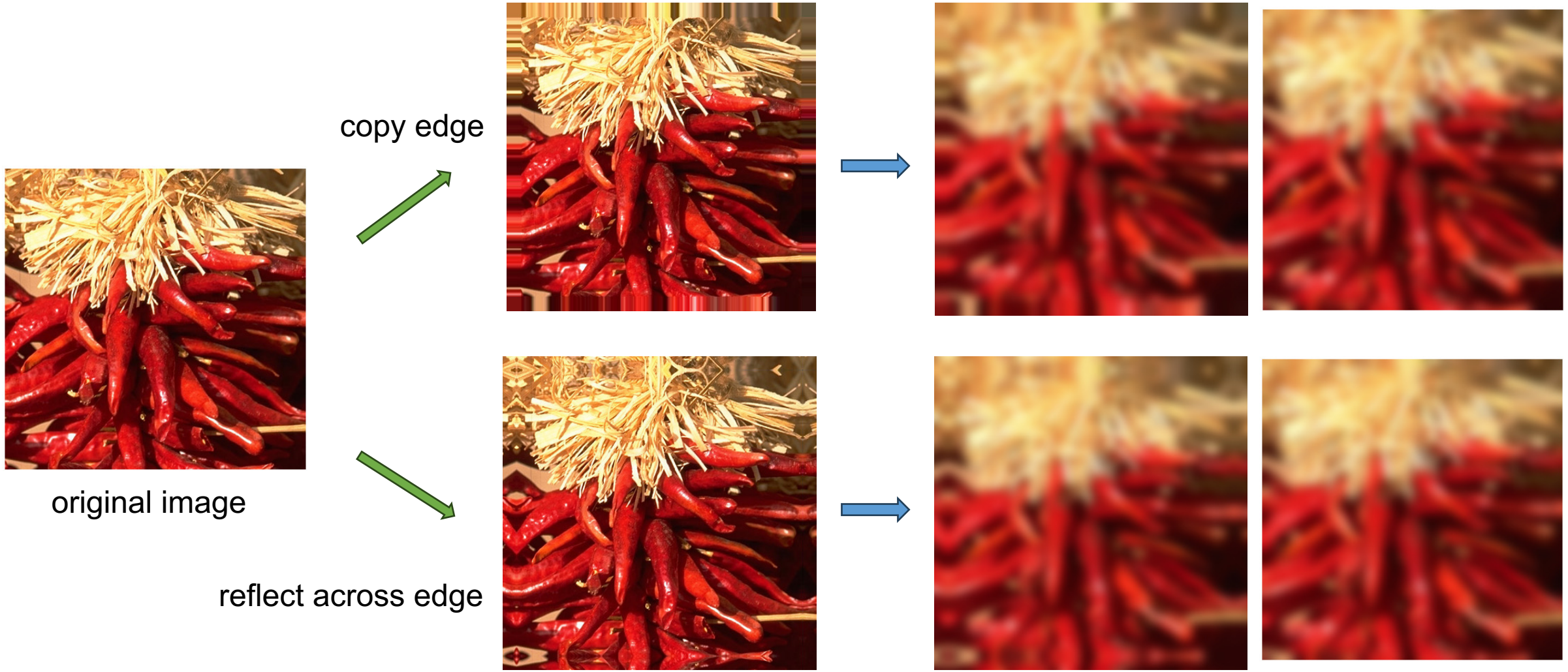
If the filter window falls off the edge of the image, we can pad the image




Source: S. Marschner

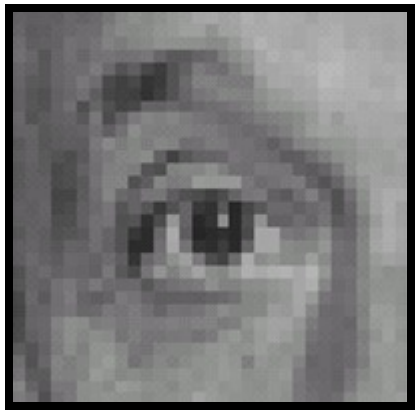
# Dealing with Edges

If the filter window falls off the edge of the image, we can pad the image




Source: S. Marschner

# Practice with Linear Filters



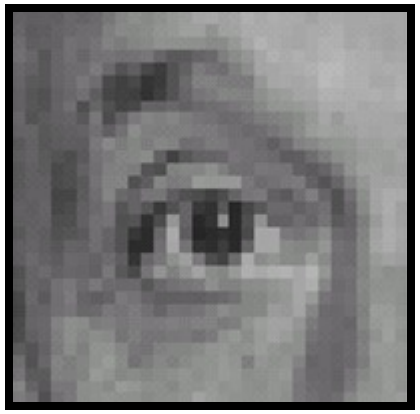
Original

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

?

Source: D. Lowe

# Practice with Linear Filters



Original

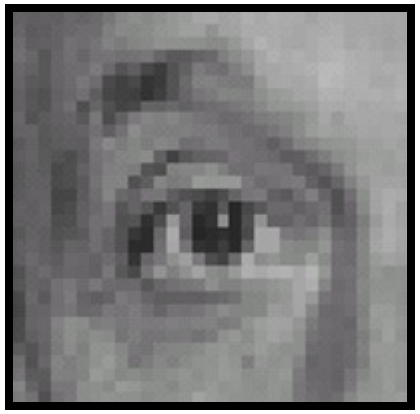
| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |



Filtered
(no change)

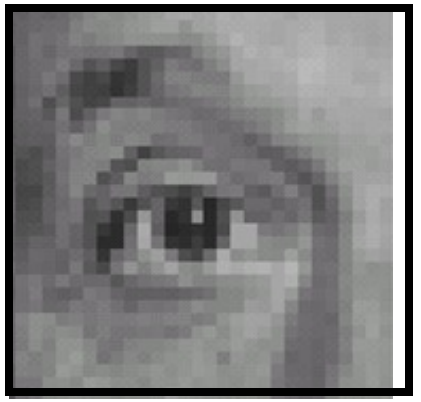
Source: D. Lowe

# Practice with Linear Filters



Original

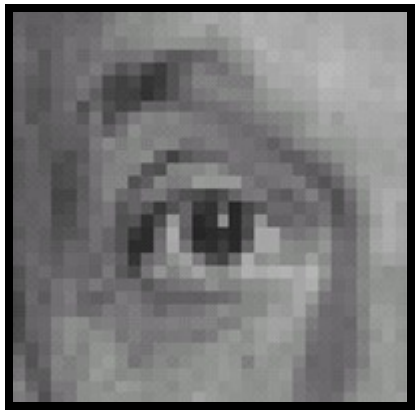
| | | |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 0 |
| 0 | 0 | 0 |



Shifted *left*
By 1 pixel

Source: D. Lowe

# Practice with Linear Filters



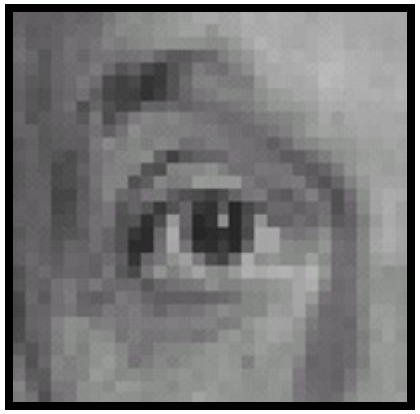
Original

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

?

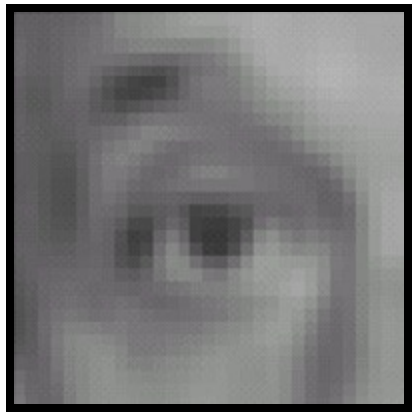
Source: D. Lowe

# Practice with Linear Filters



Original

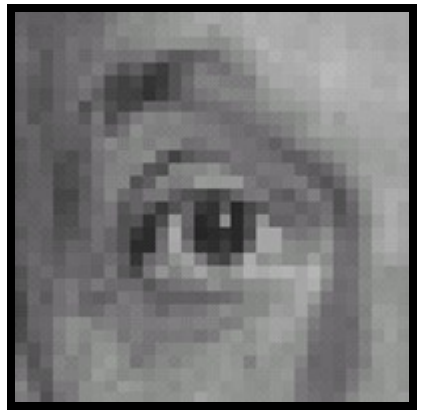
$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Blur (with a box filter)

Source: D. Lowe

# Practice with Linear Filters



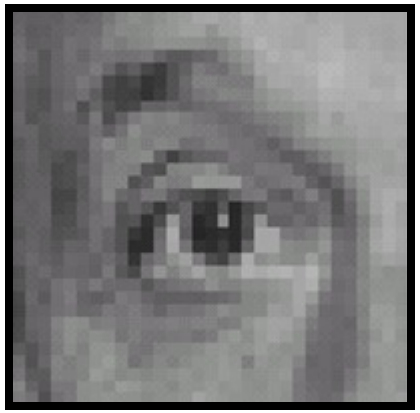
Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad ?$$

(Note that filter sums to 1)

Source: D. Lowe

# Practice with Linear Filters



Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



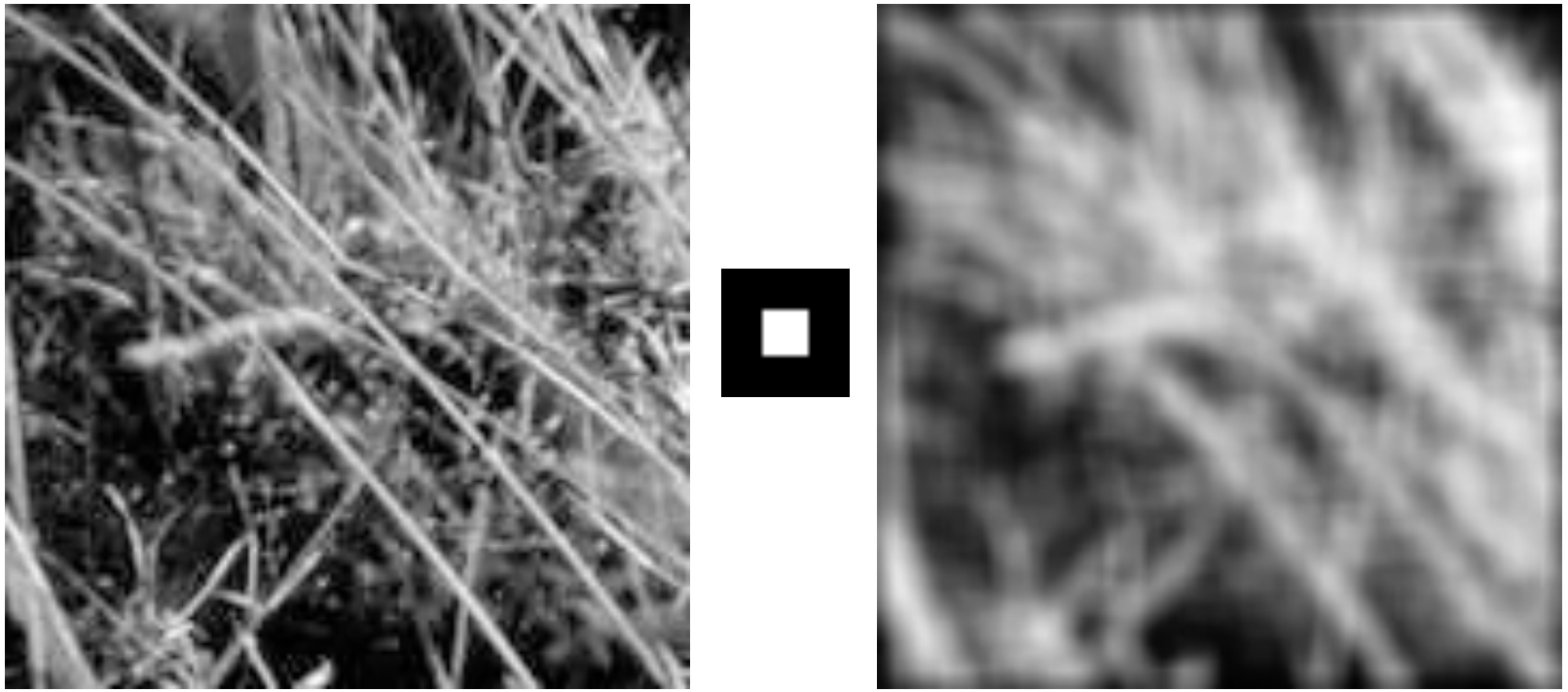
**Sharpening filter:** Accentuates differences with local average

Source: D. Lowe

# Smoothing and Sharpening

# Revisit: Smoothing with Box Filter

- What is unsatisfactory about giving every pixel inside a square neighborhood equal weight?
- What's the solution?

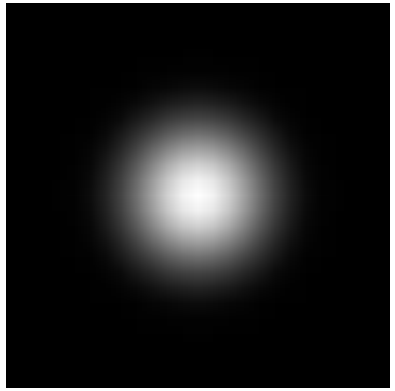


Source: D. Forsyth

# Revisit: Smoothing with Box Filter

- What is unsatisfactory about giving every pixel inside a square neighborhood equal weight?

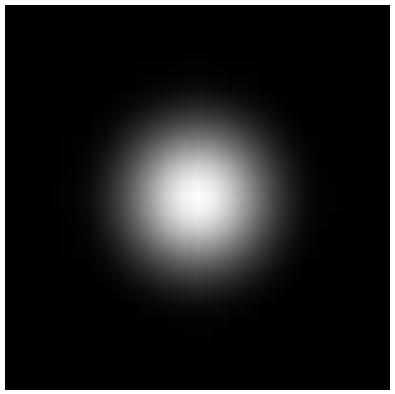
- What's the solution?
  - Weight nearby pixels more strongly than distant pixels, and vary the weights smoothly with distance

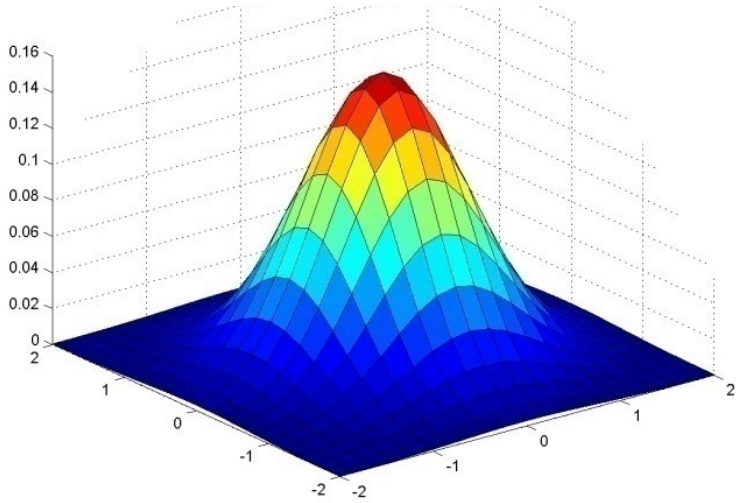


"fuzzy blob"

Credit to Lana Lazebnik

# Gaussian Kernel






**Continuous Gaussian function**

$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)}$$

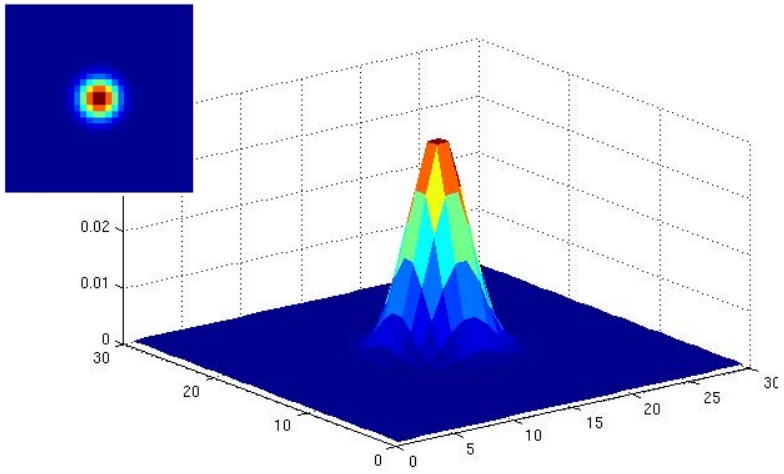
| | | | | |
|---|---|---|---|---|
| 0.003 | 0.013 | 0.022 | 0.013 | 0.003 |
| 0.013 | 0.059 | 0.097 | 0.059 | 0.013 |
| 0.022 | 0.097 | 0.159 | 0.097 | 0.022 |
| 0.013 | 0.059 | 0.097 | 0.059 | 0.013 |
| 0.003 | 0.013 | 0.022 | 0.013 | 0.003 |

**5x5 discrete Gaussian kernel**
sample->truncate->normalize

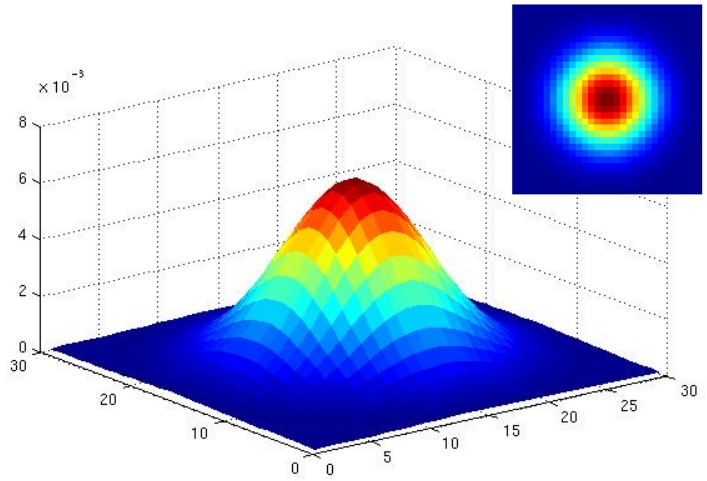
Credit to C. Rasmussen

# Gaussian Kernel

$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)}$$




σ = 2 with 30 x 30 kernel



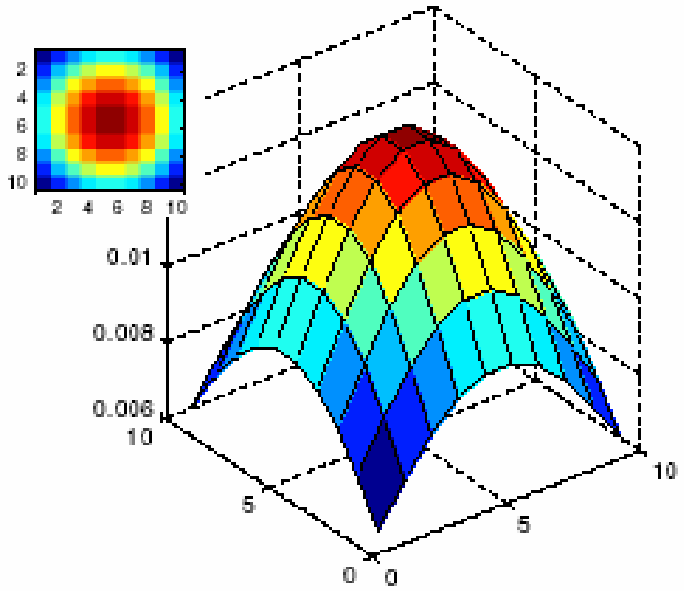

σ = 5 with 30 x 30 kernel

Standard deviation $\sigma$: determines extent of smoothing

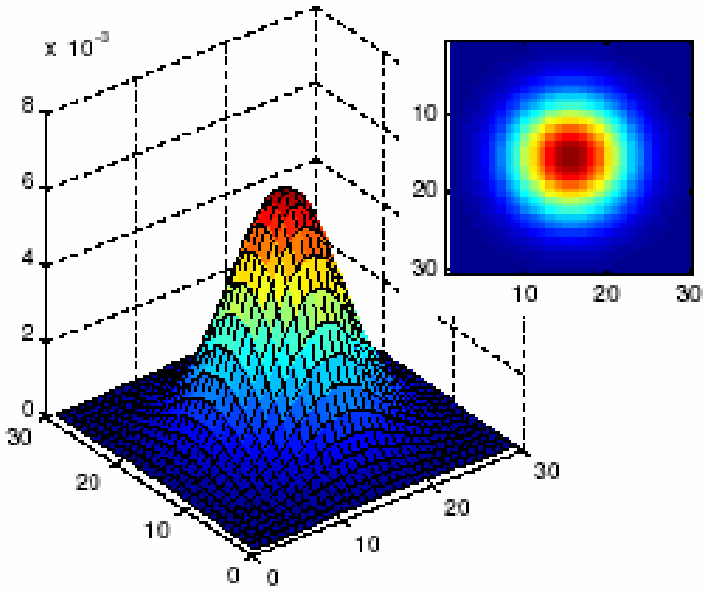
Source: K. Grauman

# Gaussian Kernel Width

- The Gaussian function has infinite support, but discrete filters use finite kernels




σ = 5 with 10x10 kernel

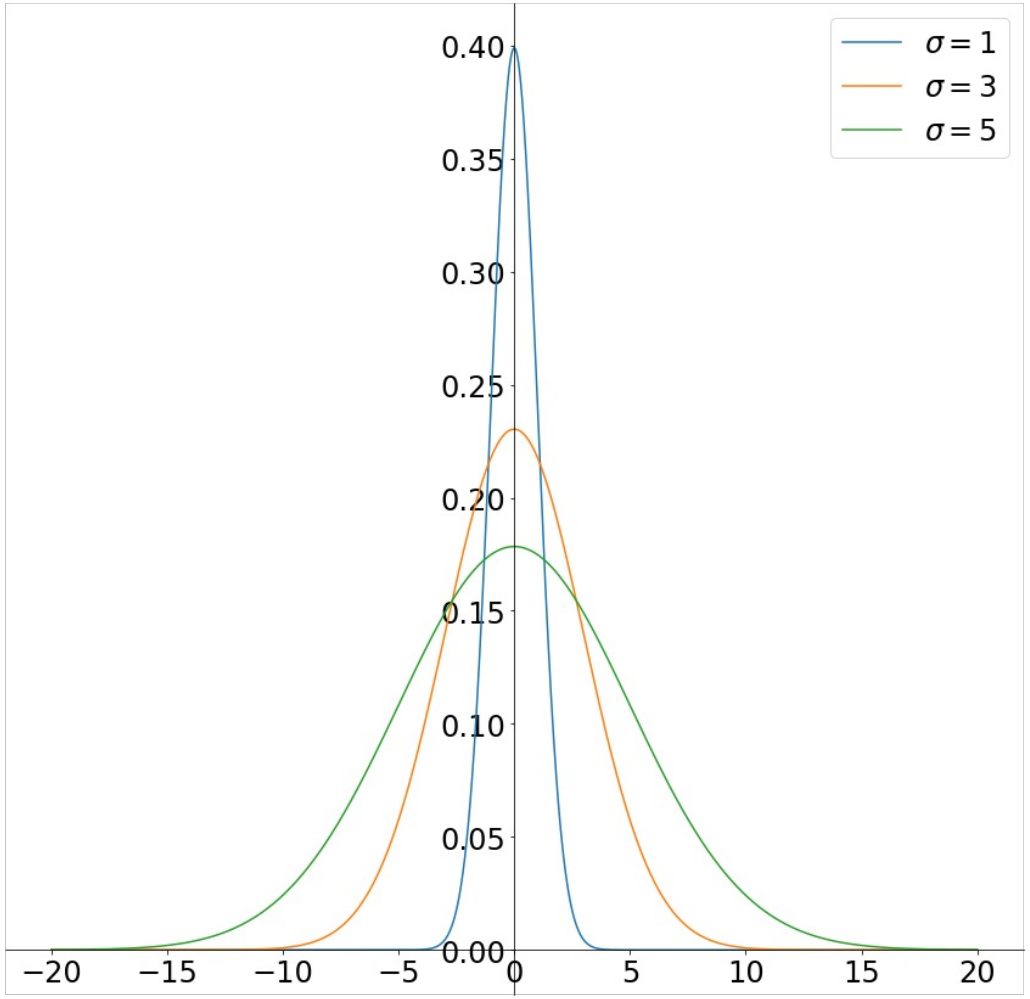



σ = 5 with 30x30 kernel

Source: K. Grauman

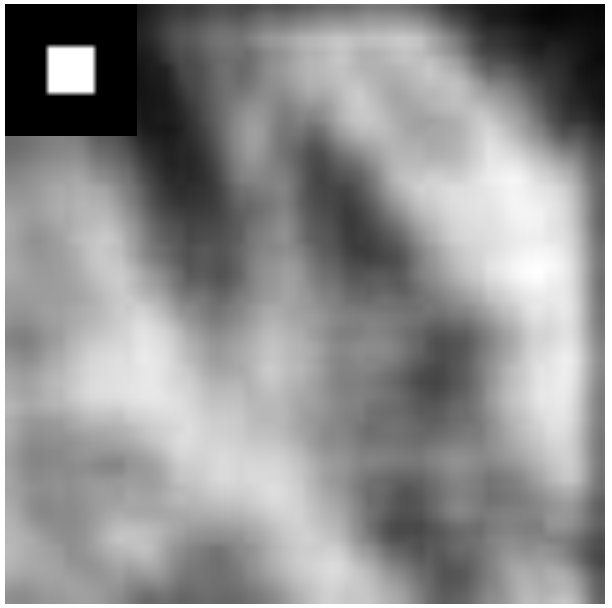
# Choosing Kernel width

- Rule of thumb: set filter half-width to about $3\sigma$

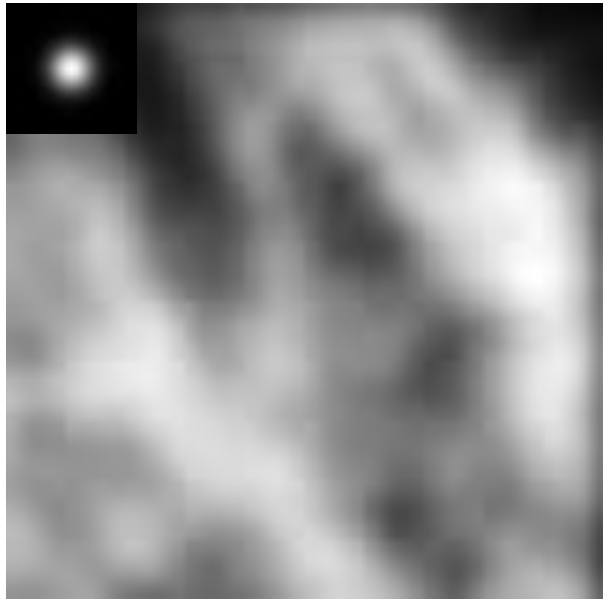


# Gaussian vs. Box Filtering





box



Gaussian

**Gaussian weights vary smoothly and give more emphasis to nearby pixels**

F&P, Figure 4.1

# Gaussian Filters

- Remove high-frequency components from the image, serving as a *low-pass* filter

- Convolution with self is another Gaussian
  - Repeating smoothing with a small-$\sigma$ kernel leads to the same result as a larger-$\sigma$ kernel would have
  - Convolving two times with Gaussian kernel with std. dev. $\sigma$ is same as convolving once with kernel with std. dev. $\sqrt{2}\sigma$

$$G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2+\sigma_1^2}}$$

Modified from K. Grauman

# Separability of the Gaussian filter

- The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y.

- In this case the two functions are the (identical) 1D Gaussian.

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)}$$
$$= \left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}\right)\left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{y^2}{2\sigma^2}}\right)$$

continuous case

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

discrete example

Modified from D. Lowe

# Why is separability useful?

- Separability means that a 2D convolution can be reduced to two 1D convolutions (one along rows and one along columns)

- What is the complexity of filtering an $n \times n$ image with an $m \times m$ kernel?
  - $O(n^2 m^2)$

- What if the kernel is separable?
  - $O(n^2 m)$

Credit to Lana Lazebnik

# Sharpening

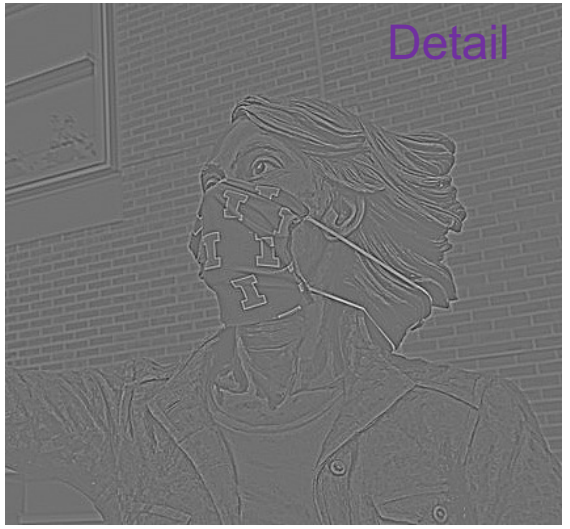
What does blurring take away?




–



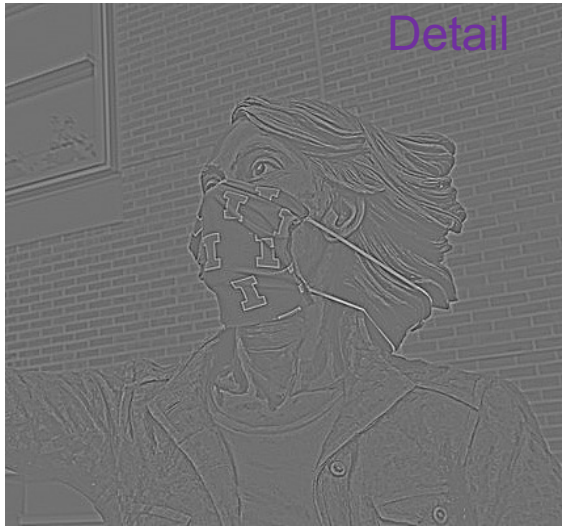

=




Let's add it back in.




$+\alpha$




=




Credit to S. Gupta

# Denoising

The right filter depends on the noise

# Different Types of Noise



Original



Salt and pepper noise



Impulse noise



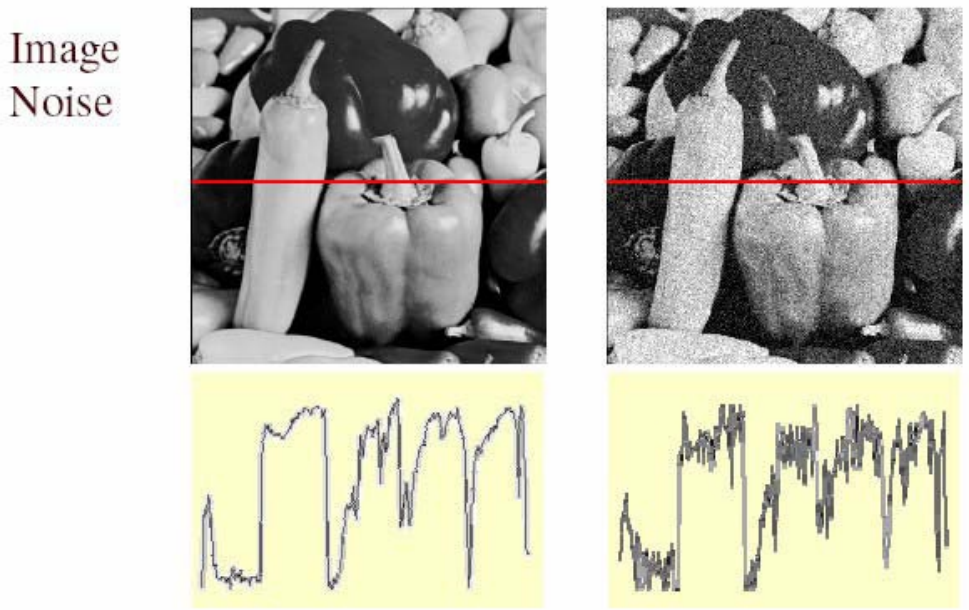
Gaussian noise

- **Salt and pepper noise**: contains random occurrences of black and white pixels
- **Impulse noise:** contains random occurrences of white pixels
- **Gaussian noise**: variations in intensity drawn from a Gaussian normal distribution

Source: S. Seitz

# Gaussian Noise

- Mathematical model: sum of many independent factors
- Good for small standard deviations
- Assumption: independent, zero-mean noise

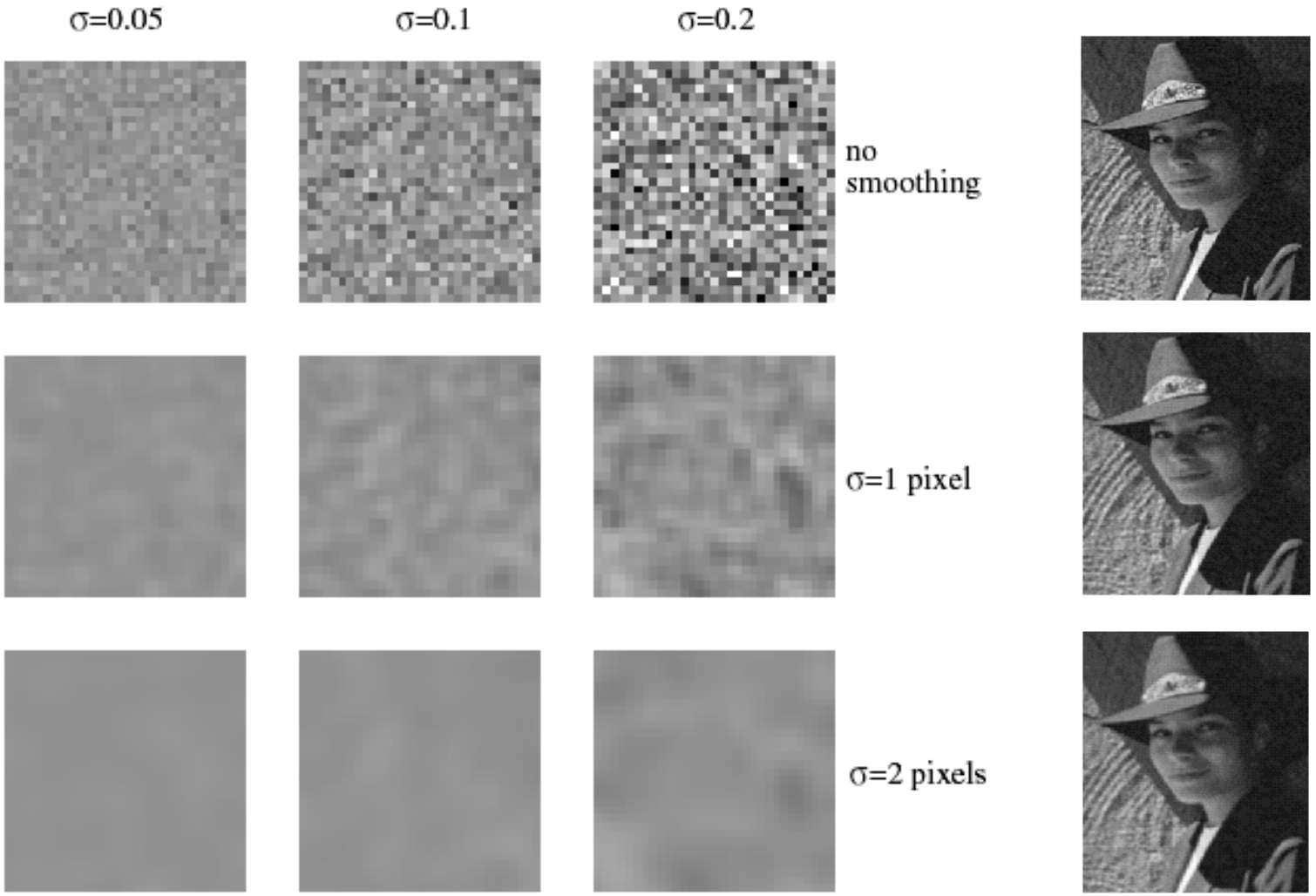



$$f(x,y) = \overbrace{\bar{f}(x,y)}^{\text{Ideal Image}} + \overbrace{\eta(x,y)}^{\text{Noise process}}$$

Gaussian i.i.d. ("white") noise:
$\eta(x,y) \sim \mathcal{N}(\mu,\sigma)$

Source: M. Hebert

# Reducing Gaussian Noise




Smoothing with larger standard deviations suppresses noise, but also blurs the image

Credit to Lana Lazebnik

# Reducing Salt-and-Pepper Noise

3x3



5x5



7x7



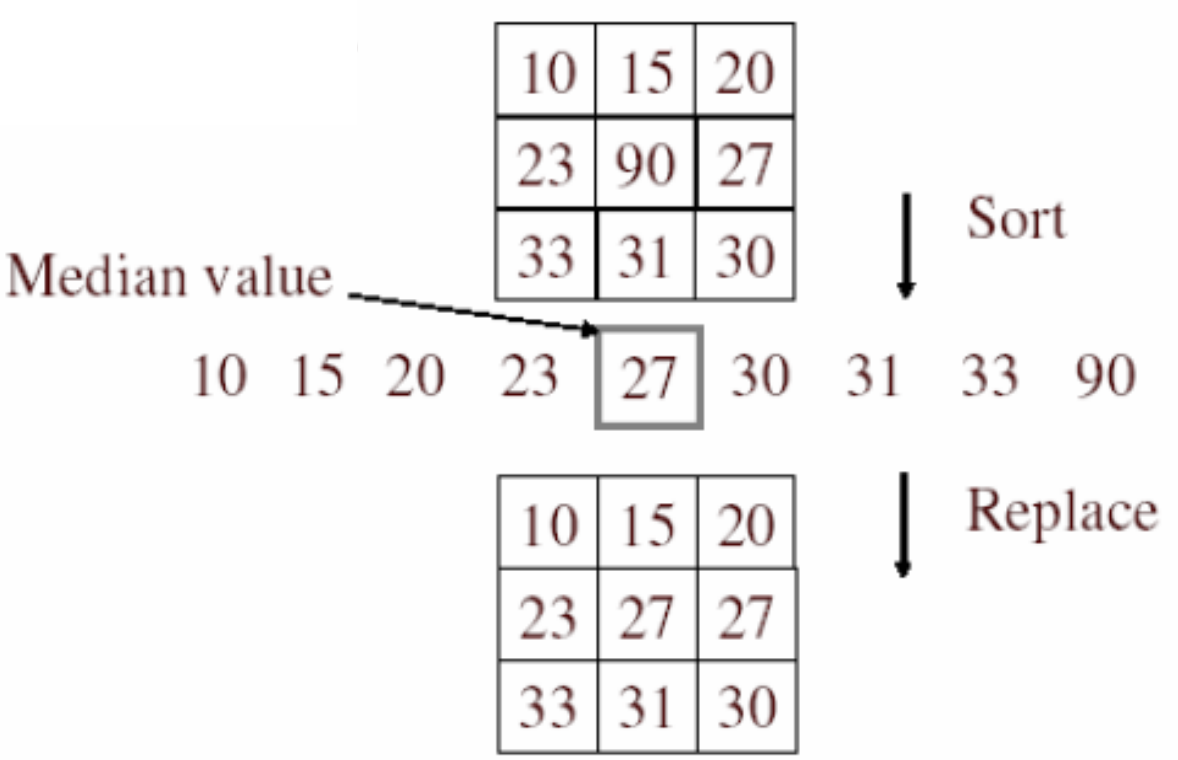
What's wrong with the results?

Credit to Lana Lazebnik

# Alternative Idea: Median Filtering

- A **median filter** operates over a window by selecting the median intensity in the window

- Is median filtering linear?




Source: K. Grauman

# Is median filtering linear?

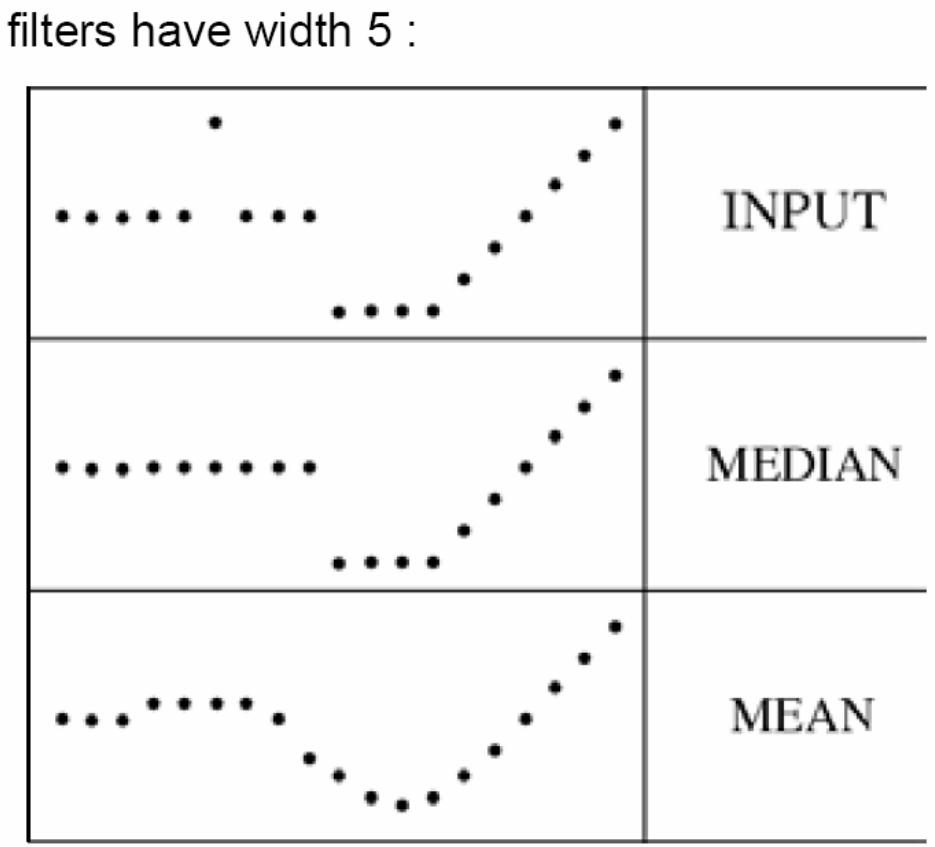
- Let's check linearity

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 2 & 2 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Credit to Lana Lazebnik

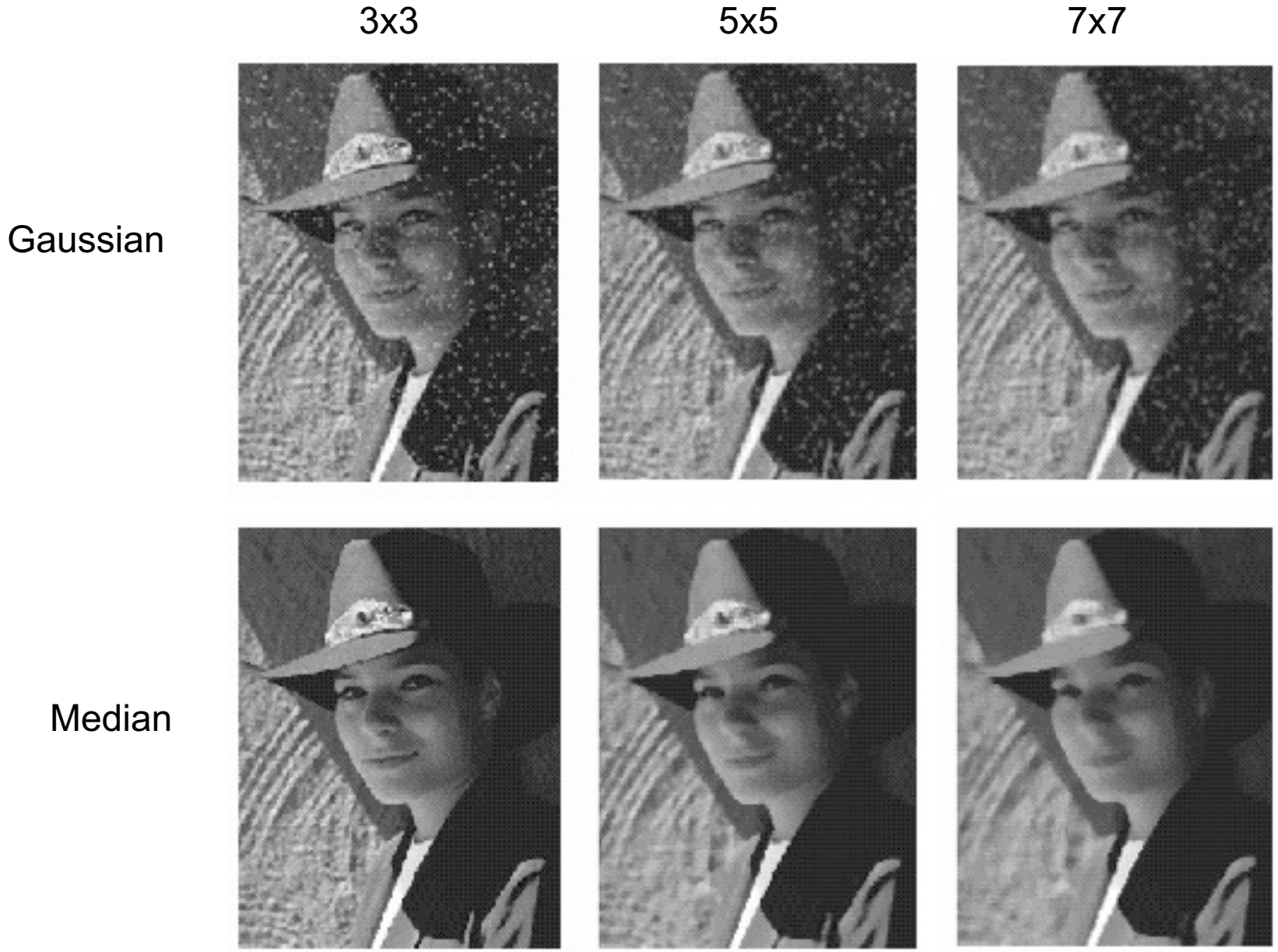
# Median Filter

- What advantage does median filtering have over Gaussian filtering?
  - Robustness to outliers




Source: K. Grauman

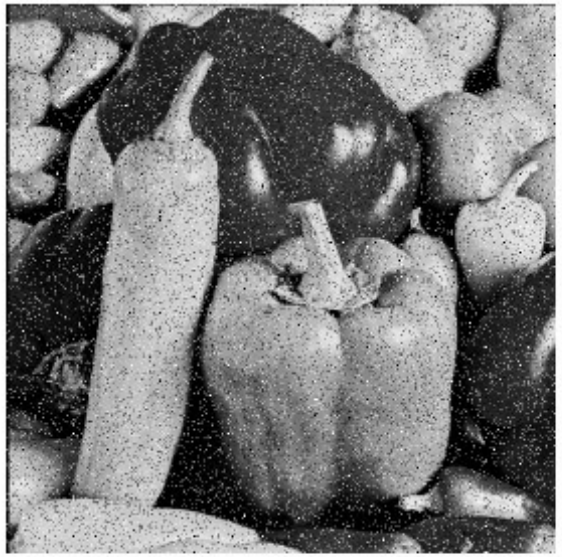
# Gaussian vs. Median Filtering

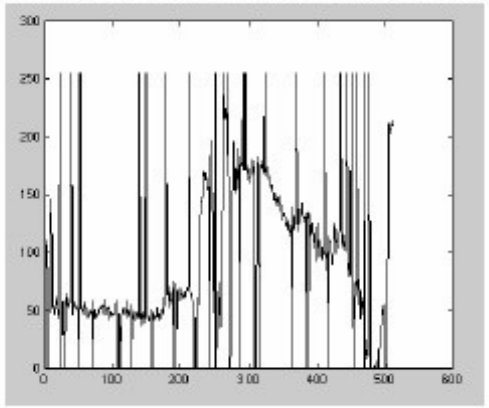



Credit to Lana Lazebnik

# Median Filter on Salt-and-Pepper Noise

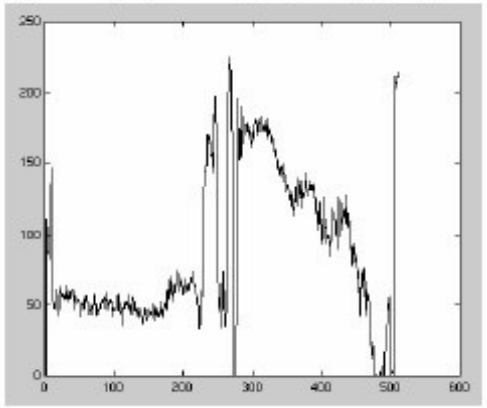
Salt-and-pepper noise






Median filtered






Source: M. Hebert

# Summary

- A kernel defines a local spatial operation
- Convolution flips the kernel; correlation does not
- Gaussian smoothing: $G_\sigma$
- Sharpening: $I_{sharp} = I + \alpha(I - G_\sigma * I)$
- Median filtering (nonlinear)

# Reading List

- **Szeliski, *Computer Vision: Algorithms and Applications*, 2e.**
  - Ch. 3 Image Processing
  - Ch. 3.2 Linear Filtering
  - Ch. 3.3 Non-linear Filtering
- **Forsyth & Ponce, *Computer Vision: A Modern Approach*, 2e.**
  - Ch. 4 Linear Filters
- OpenCV tutorial